



Angular

INTRODUCCION

¿Qué es Angular?

Angular es un **framework** de desarrollo de aplicaciones de código abierto y de alto rendimiento creado por **Google**. Proporciona una estructura y conjunto de herramientas para construir aplicaciones web dinámicas y escalables. Angular sigue el patrón de diseño Modelo-Vista-Controlador (**MVC**)

Tiene todo lo que necesitas

- Gestor de estado
- Enrutamiento
- Reactividad
- Peticiones HTTP
- Directivas
- Etc.

Es Multiplataforma

- Web: SPA, SSR, SSG Enrutamiento
- Móvil: ionic, NativeScript
- Desktop: Electrón

Una aplicación de Angular se organiza normalmente en componentes, cada uno de los cuales puede tener/utilizar

- **HTML** como lenguaje de plantillas → Estructura de la interfaz.
- **CSS** → Apariencia de la interfaz. (opcional)
- **TypeScript** (principalmente) como lenguaje de programación → Lógica y comportamiento

NOTA: No hay que confundir Angular con AngularJS: AngularJS es la tecnología antigua basada en JavaScript; Angular es su sucesor moderno basado en TypeScript

Bloques Fundamentales

Componentes

Rutas

Directivas

Servicios

Modulos

Pipes

Bloques Fundamentales

Componentes

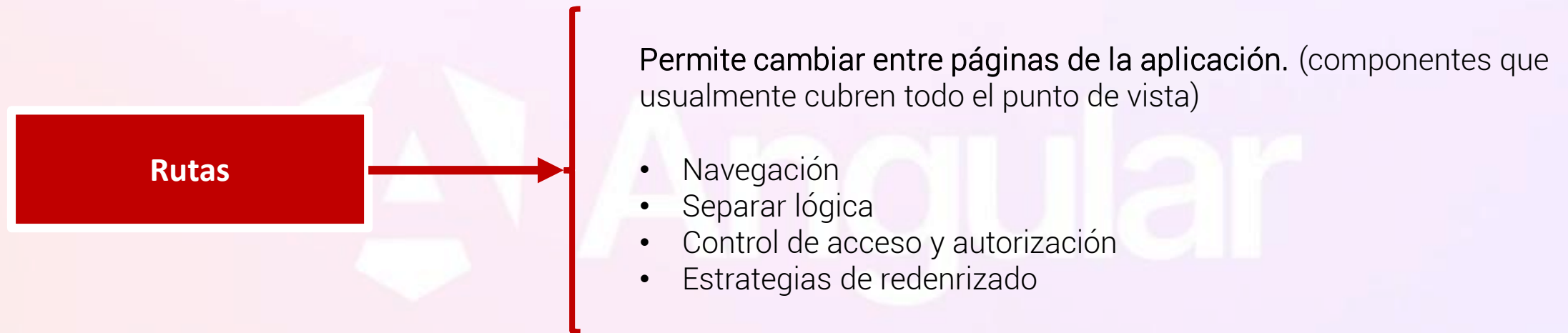
Es una pieza que representa una parte de la interfaz de usuario.

- Lógica (TypeScript)
- Estilos (CSS, SASS, etc) - opcional
- Plantilla HTML

```
import { Component } from '@angular/core';

@Component({
  selector: 'mi-componente',
  templateUrl: './mi-componente.component.html',
  styleUrls: ['./mi-componente.component.css']
})
export class MiComponenteComponent {
  // Lógica y datos aquí
}
```

Bloques Fundamentales



Bloques Fundamentales

Directivas

Modifican el comportamiento de un elemento HTML

- Atributo – ngClass, ngStyle, ...
- Estructurales – ngIf, ngFor, ngSwitch, (@ = nuevo control flow)
- Componentes – Personalizadas en el propio html (templates)

¿Para qué sirven?

- **Reutilizar código:** Aplicar una misma lógica de diseño o interacción en múltiples partes de la aplicación.
- **Controlar la vista:** Mostrar u ocultar secciones de la página de forma dinámica.
- **Manipular estilos:** Adaptar colores, clases o atributos basados en datos en tiempo real

Bloques Fundamentales

Servicios

Clase de TypeScript que encapsulan lógica de negocio, datos o funciones compartidas y centralizan su acceso.

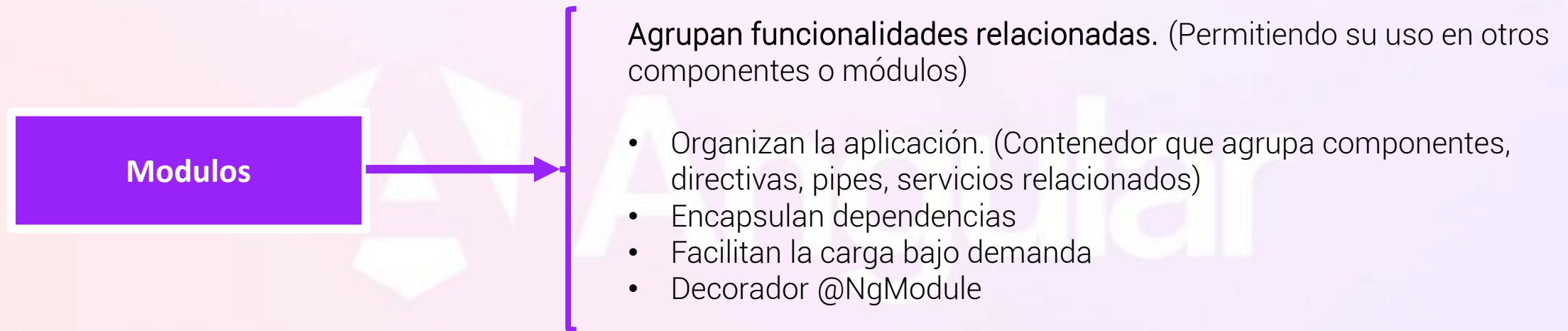
- Gestión de datos (separación de código)
- Reutilización de código
- Inyección de dependencias

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root' // Disponible en toda la aplicación
})
export class MiServicio {
  private datos: string[] = ['Elemento 1', 'Elemento 2'];

  obtenerDatos() {
    return this.datos;
  }
}
```

Bloques Fundamentales



Estructura de NgModule

- **Declarations:** Componentes, directivas y pipes que pertenecen a este módulo.
- **Imports:** Otros módulos necesarios para que este funcione
- **Providers:** Servicios que estarán disponibles globalmente en el ámbito del módulo.
- **Exports:** Elementos de este módulo que otros módulos pueden usar.

Bloques Fundamentales



Pipes más comunes integrados en Angular

- `uppercase`: Convierte un texto a letras mayúsculas (`{{ nombre | uppercase }}`).
- `lowercase`: Convierte un texto a letras minúsculas.
- `date`: Da formato a las fechas (`{{ fecha | date:'shortDate' }}`).
- `currency`: Muestra valores numéricos como monedas (`{{ precio | currency }}`).
- `json`: Convierte un objeto en un string formato JSON para depurar.
- `async`: Lee datos directamente desde una promesa o un observable

Mitos y Realidades sobre Angular

MITOS

- Angular es mejor que React, Vue, Solid, ...
- Angular es mas ordenado que React, Vue, Solid, ...
- Las aplicaciones de Angular son muy pesadas.
- Angular no es SEO friendly.
- Angular no soporta patrones como Redux
- Solo puedo correr código TypeScript

REALIDAD

- Es más complicado de aprender.
- Libera versiones frecuentemente (aprox. Cada 6 meses)
- Angular 2,4,5..,20... son el mismo Angular



Diferencias entre Angular antiguo y moderno

- Angular antiguo estaba principalmente organizado alrededor de módulos y RxJS
- Angular moderno está orientado a componentes standalone, Signals, APIs funcionales y una reactividad más granular..

Idea clave.

- Angular no ha cambiado de arquitectura; ha ido **evolucionando** hacia una plataforma más simple, reactiva, eficiente y menos dependiente de NgModule y Zone.js
- Si vienes de Angular antiguo **NO tienes que aprender Angular desde cero**. Los conceptos siguen siendo válidos; lo que cambia principalmente es la forma recomendada de estructurar y hacer reactiva la aplicación.

Aspecto	Angular antiguo	Angular moderno
Arquitectura	Basada en módulos, componentes y servicios	Componentes standalone como enfoque principal
Componentes	@Component + NgModule	@Component({ standalone: true }) o standalone por defecto en versiones recientes
Estado reactivo	Principalmente RxJS + Observable	Signals + RxJS cuando es necesario
Detección de cambios	Principalmente Zone.js	Signals y nuevas estrategias de detección más eficientes; Zone.js puede ser opcional
Plantillas	*ngIf, *ngFor, *ngSwitch	Nuevas estructuras: @if, @for, @switch
Inyección de dependencias	providers en módulos/componentes	DI más flexible y orientada a standalone
Carga de código	NgModules y lazy loading	Lazy loading de rutas/componentes de forma más directa
Formularios	Template-driven o Reactive Forms	Siguen disponibles; se añaden APIs más modernas progresivamente
Rendimiento	Bueno, pero con más trabajo manual	Mejoras en build, renderizado, hidratación y reactividad
Sintaxis	Más verbosa	Más declarativa y concisa
Estado global	RxJS, servicios, NgRx, etc.	Signals, servicios, RxJS, NgRx, etc.
Testing	Jasmine/Karma muy habituales	Mayor flexibilidad y herramientas modernas
Build	Webpack y configuraciones tradicionales	Esbuild/Vite y herramientas de compilación más modernas



Diferencias entre Angular antiguo y moderno

Evolución esquemática

ANGULAR ANTIGUO

*

- NgModules
- Components + Directives
- RxJS / Observables
- *ngIf / *ngFor
- Zone.js
- Webpack

ANGULAR MODERNO

*

- Standalone Components
- Signals
- @if / @for / @switch
- Zoneless opcional
- SSR + Hydration
- Lazy loading simplificado
- Build moderno (esbuild/Vite)

Angular y TypeScript

¿Qué es TypeScript?

- TypeScript un superconjunto de JavaScript que agrega características adicionales como el **tipado estático**, interfaces, clases y módulos.
- TypeScript se transpila y compila a JavaScript estándar
- Ampliamente utilizado en el desarrollo de aplicaciones Angular debido a su estrecha integración con el framework



!! Si sabes JavaScript conoces mínimo el 80% de TypeScript !!

La principal diferencia entre TypeScript y JavaScript.

- TypeScript tiene **tipado estatico**.
 - Permite declarar y verificar los tipos de variables, parámetros y resultados de funciones durante el desarrollo (ayuda a detectar errores en el editor antes de la ejecución del código). Mientras que JavaScript usa **tipado dinámico**.
- TypeScript admite características adicionales de programación orientada a objetos, como **clases, interfaces y herencia de tipos**, que no dispone JavaScript.
 - Estas características hacen que el código TypeScript sea más estructurado y mantenible en proyectos de gran escala.

JavaScript VS TypeScript

```
function calculateTax( products, taxRate ) {  
  
  let total = 0;  
  
  products.forEach( product => {  
    total += product.price;  
  } );  
  
  return total * taxRate;  
}
```



- ¿Cómo luce un producto?
- ¿Qué propiedades tiene?
- ¿Qué debería de tener?

¿Ese código funciona?



```
interface Product {  
  name: string;  
  amount: number;  
}  
  
function calculateTax( products: Product[], taxRate: number ) {  
  
  let total = 0;  
  
  products.forEach( product => {  
    total += product.price;   
  } );  
  
  return total * taxRate;  
}
```

Property 'price' does not exist on type 'Product'.

- Error en el editor
- Sabemos cómo luce
- Métodos y propiedades de los objetos



Pasos para empezar a trabajar con Angular

1. Instalar Node.js y npm

- Descarga e instala Node.js en tu ordenador.
- Esto incluye **npm** (el gestor de paquetes), necesario para descargar herramientas de Angular.
- Comprueba que se instalaron bien abriendo tu terminal y escribiendo: `node -v` y `npm -v`. [\[1\]](#)

2. Instalar Angular CLI

- [Lista compatibilidad de versiones](#)
- Escribe el siguiente comando en tu terminal para instalar la herramienta oficial de Angular de forma global:
→ `npm install -g @angular/cli`
- Visualiza el proceso inicial de instalación de Node.js y Angular CLI en la terminal

3. Crear un nuevo proyecto

- Abre tu terminal y ve a la carpeta donde quieras guardar tu trabajo.
- Ejecuta el comando para crear una nueva aplicación (cambia `mi-app` por el nombre que prefieras):
→ `ng new mi-app`



Ojo con la compatibilidad de versiones

1. La Trinidad de Compatibilidad (Node, TypeScript y Angular)

Angular depende directamente de versiones específicas de Node.js y TypeScript. Si usas una versión de Node demasiado nueva o antigua para tu versión de Angular, el proyecto ni siquiera compilará.

- **Node.js:** El entorno de ejecución. Angular suele requerir versiones **LTS (Long Term Support)** de Node.
- **TypeScript:** El lenguaje en el que escribes Angular. Cada versión de Angular tiene un rango estricto de TypeScript que soporta.
- **Angular CLI:** La herramienta de comandos. Debe coincidir en versión mayor con el framework de Angular de tu proyecto.

NOTA: Consulta siempre la [Tabla Oficial de Compatibilidad de Angular](#) para verificar qué versión le corresponde a cada una.

2. Control de Versiones del Proyecto (package.json)

Dentro de tu proyecto, el archivo `package.json` gestiona las dependencias.

- **^17.0.0 (Caret):** Permite actualizaciones de versiones *minor* y *patch* (ej. instalaría la 17.1.0 o 17.0.2, pero nunca la 18.0.0). Defecto.
- **~17.0.0 (Tilde):** Solo permite actualizaciones de *patch* (ej. instalaría la 17.0.1, pero no la 17.1.0).
- **17.0.0 (Fija):** Instala exactamente esa versión. Es la más segura para evitar sorpresas en equipos de trabajo.

3. Herramientas recomendadas para el día a día

- **NVM (Node Version Manager):** Es imprescindible o al menos muy útil.
 - Te permite cambiar de versión de Node.js con un solo comando en tu terminal.
 - Esto es vital si trabajas en varios proyectos de Angular creados en años diferentes.
- **package-lock.json:** Nunca lo borres ni lo ignores en Git.
 - Este archivo asegura que todo tu equipo de desarrollo (y el servidor de producción) instale exactamente las mismas sub-dependencias.

Instalación y Recursos

AngularCLI

- Documentación [oficial de Angular CLI](#)
- Instalación (Ejecutar como administrador)

```
npm install -g @angular/cli
```

Recursos y Aplicaciones a Utilizar

1. [NVM: Gestor versiones node.js \(Recomendado\)](#)
2. [Node JS](#)
3. [VSCode - Visual Studio Code](#)
4. [Google Chrome](#) (u otro navegador basado en Chromium)
5. [Postman](#)
6. [Git](#)

Extensiones recomendadas de VSCode

- [Angular Language Service](#)
- [Angular Snippets](#)
- [Angular Schematics](#)
- [Angular 2 Inline](#)
- [Auto Close Tag](#)
- [Auto import](#)
- [Auto Rename Tag](#)
- [Error Lens](#)
- [Paste JSON as Code](#)
- [TypeScript Importer - optional](#)
- [Editor Config for VSCode](#)
- [Better Comments](#)
- [Tailwind CSS IntelliSense](#)
- [Pretty TypeScript Errors](#)

Extensiones recomendadas de Chrome

- [Angular DevTools](#)
- [Json Viewer Awesome](#)

NVM: Gestión de proyectos y versiones de Angular

NVM

Gestor de Versiones de Node.js

Gestiona fácilmente múltiples versiones de Node.js

Comenzar

Descargar



[Ir a presentación básica de NVM](#)

FIN BLOQUE 1



(INTRODUCCION)

Router en Angular (I)

El **Router** es el encargado de **gestionar la navegación** entre las diferentes vistas o componentes de una aplicación Angular y determina que componente debe mostrarse para cada URL.

1. Router → (GESTIONAR)

- Controla la navegación y las rutas definidas.
- `const routes: Routes = [ { path: 'inicio', component: InicioComponent }, { path: 'usuarios', component: UsuariosComponent } ];`

```
src > app > TS app.routes.ts
1  import { Routes } from '@angular/router';
2  import { AppComponent } from './app.component';
3  import { CounterPageComponent } from './pages/counter/counter-page.component';
4  import { HeroPageComponent } from './pages/hero/hero-page.component';
5  import { Personas1PageComponentComponent } from './pages/personas/personas1-page-component.component';
6
7  export const routes: Routes = [
8
9      { path: '', redirectTo: 'counter', pathMatch: 'full' },
10
11     { path: 'counter', component: CounterPageComponent },
12
13     { path: 'hero', component: HeroPageComponent },
14
15     { path: 'personas', component: Personas1PageComponentComponent }
16
17     // redirigir cuando en la barra de navegacion se ponga una ruta que no existe.
18     // Se puede poner un componente de error en vez de redirigir a counter
19     //{ path: '**', redirectTo: 'counter' }
20     //{ path: '**', component: PageNotFoundComponent }
21 ];
22
```

Elemento	Función
Router	Gestiona la Navegación
Routes	Definir las rutas disponibles
RouterLink	Permite navegar desde HTML
RouterLinkActive	Marcar la ruta activa
RouterOutlet	Muestra el componente

Router en Angular (II)

El **Router** es el encargado de **gestionar la navegación** entre las diferentes vistas o componentes de una aplicación Angular y determina que componente debe mostrarse para cada URL.

1. Router → (GESTIONAR)

- Controla la navegación y las rutas definidas.
- `const routes: Routes = [ { path: 'inicio', component: InicioComponent }, { path: 'usuarios', component: UsuariosComponent } ];`

2. RouterLink → (NAVEGAR) ¿A dónde quiero navegar? .

- Permite navegar entre componentes/vistas de una aplicación Angular sin recargar la página. (SPA)
- Indica a que ruta queremos ir
- Ejemplo: `<a routerLink="/usuarios">Usuarios</a>`

3. RouterLinkActive → (MARCAR RUTA ACTUAL) ¿Qué enlace esta seleccionado?.

- Permite Aplicar una clase cuando la ruta este activa.
- Angular añade automáticamente la clase css "activo"
- Ejemplo: `<a routerLink="/usuarios" routerLinkActive="activo">Usuarios</a>`

4. RouterOutlet → Mostrar el contenido (MOSTRAR).

- Indica donde se rederiza el componente asociado a la ruta
- `<router-outlet></router-outlet>`

FLUJO COMPLETO

1. *CLICK*
2. *RouterLink*
3. *Router*
4. *RouterOutlet*
5. *Componente*
6. *RouterLinkActivate*
7. *Marca el enlace como activo*